## Introduction to Algorithms and Programming Languages:

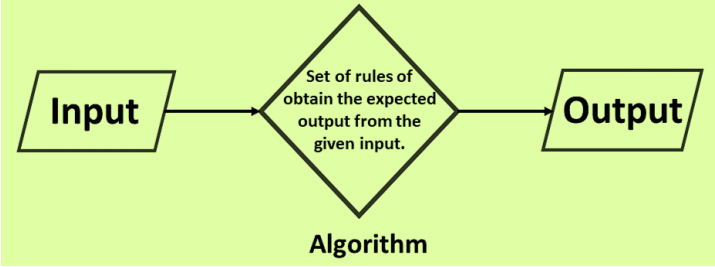
An algorithm is a popular term that come across in a variety of contexts, including computer programming, mathematics, and even our everyday life. In normal words, an algorithm describes a mathematical procedure or way to resolve an issue using a finite or definite number of steps.

According to the world of coding and computers, Algorithms are sets of instructions that assist us to resolve a problem within a definite or finite number of steps.

To better understand this definition by assuming it is like a recipe. A baker knows that there are many ways to make a cookie, but by following a recipe he knows to first preheat the oven. Next, he measures out the flour, and then adds chips, butter, etc. until the cookies are ready.

An algorithm is a step-by-step procedure that defines a set of instructions that must be carried out in a specific order to produce the desired result. It is not the entire program or code; it is simple logic to a problem represented as an informal description in the form of a flowchart or pseudocode.

Algorithm is a set of unambiguous instructions that must be followed for a computer to perform calculations or other problem-solving operations.




***Problem:*** A problem can be defined as a real-world problem for which we need to develop a program or set of instructions.

***Algorithm:*** An algorithm is defined as a step-by-step process that will be designed for a problem.

***Input:*** After designing an algorithm, the algorithm is given the necessary and desired inputs.

***Processing unit:*** The input will be passed to the processing unit, producing the desired output.

***Output:*** The outcome or result of the program is referred to as the output.

***An algorithm must possess following characteristics:***

- ***Finiteness***: An algorithm should have finite number of steps and it should end after a finite time.
- ***Input***: An algorithm may have many inputs or no inputs at all.
- ***Output***: It should result at least one output.
- ***Definiteness***: Each step must be clear, well-defined and precise. There should be no any ambiguity.
- ***Effectiveness***: Each step must be simple and should take a finite amount of time.

***Guidelines for Developing an Algorithm:***

Following guidelines must be followed while developing an algorithm:

1. An algorithm will be enclosed by START (or BEGIN) and STOP (or END).
2. To accept data from user, generally used statements are INPUT, READ, GET or OBTAIN.
3. To display result or any message, generally used statements are PRINT, DISPLAY, or WRITE.
4. Generally, COMPUTE or CALCULATE is used while describing mathematical expressions and based on situation relevant operators can be used.

**Example of an Algorithm**

***Algorithm*** *: Calculation of Simple Interest*

Step 1: Start

Step 2: Read principle (P), time (T) and rate (R)

Step 3: Calculate I = P*T*R/100

Step 4: Print I as Interest

Step 5: Stop

**Advantages of an Algorithm**

*Designing an algorithm has following advantages:*

***Effective Communication:*** Since algorithm is written in English like language, it is simple to understand step-by-step solution of the problems.

***Easy Debugging:*** Well-designed algorithm makes debugging easy so that we can identify logical error in the program.

***Easy and Efficient Coding***: An algorithm acts as a blueprint of a program and helps during program development.

***Independent of Programming Language:*** An algorithm is independent of programming languages and can be easily coded using any high level language.

**Disadvantages of an Algorithm**

*An algorithm has following disadvantages:*

1. Developing algorithm for complex problems would be time consuming and difficult to understand.
2. Understanding complex logic through algorithms can be very difficult.

**Flowcharts:**

Flowcharts are generally used in process industries to indicate the flow of the product during various stages of the process. This is a combination of an outline process chart and the flow diagram, where each operation is represented by the appropriate shape of the equipment. This gives a visual picture of the equipment as well as the operation sequence. Sometimes the equipment is represented by functional blocks, as illustrated therein. Even in computer programming, flowcharts are used to represent the series of decisions and the corresponding action taken.

A flowchart can be useful for illustrating the overall algorithm (process) graphically, particularly when learning programming.

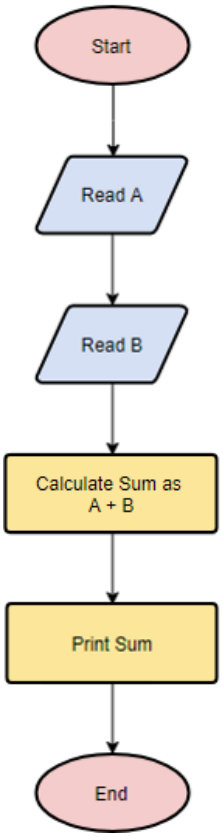
A flowchart is a graphical representation of steps. It is a tool for representing algorithms and programming logic but had extended to use in all other kinds of processes. Nowadays, flowcharts play an extremely important role in displaying information and assisting reasoning. They help us

visualize complex processes, or make explicit the structure of problems and tasks. A flowchart can also be used to define a process or project to be implemented.

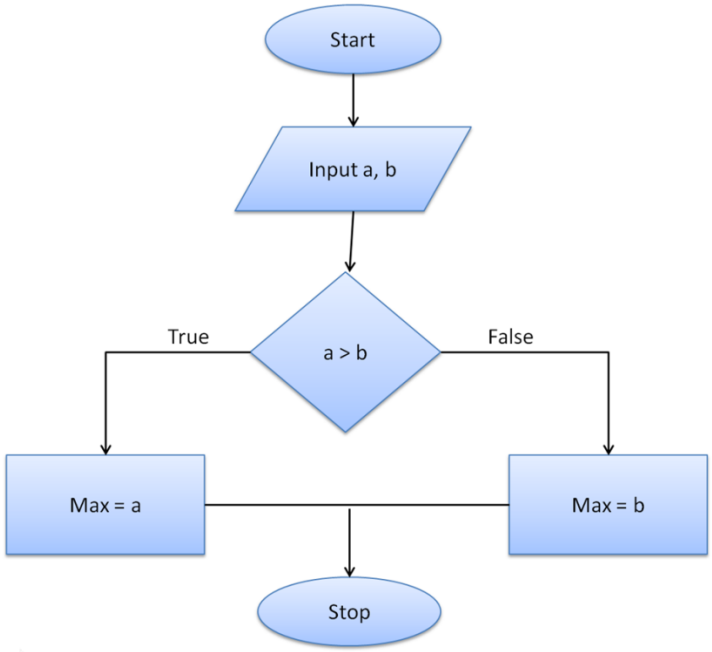
**Flowchart Symbols**

Different flowchart shapes have different conventional meanings. The meanings of some of the more common shapes are as follows:

| | | |
|---|---|---|
| **Terminator** |  | The terminator symbol represents the starting or ending point of the system. |
| **Process** |  | A box indicates some particular operation. |
| **Document** |  | This represents a printout, such as a document or a report. |
| **Decision** |  | A diamond represents a decision or branching point. Lines coming out from the diamond indicates different possible situations, leading to different sub-processes. |
| **Data** |  | It represents information entering or leaving the system. An input might be an order from a customer. Output can be a product to be delivered. |
| **On-Page Reference** |  | This symbol would contain a letter inside. It indicates that the flow continues on a matching symbol containing the same letter somewhere else on the same page. |
| **Off-Page Reference** |  | This symbol would contain a letter inside. It indicates that the flow continues on a matching symbol containing the same letter somewhere else on a different page. |
| **Delay or Bottleneck** |  | Identifies a delay or a bottleneck. |
| **Flow** |  | Lines represent the flow of the sequence and direction of a process. |




**Example 1**




**Example 2**

**Pseudocode:**

Pseudo code literally means ***'fake code'***. It is an informal and contrived way of writing programs in which you represent the sequence of actions and instructions (aka algorithms) in a form that humans can easily understand.

You see, computers and human beings are quite different, and therein lay the problem.

The language of a computer is very rigid: you are not allowed to make any mistakes or deviate from the rules. Even with the invention of high-level, human-readable languages like Java and Python, it's still pretty hard for an average human developer to program in those coding languages.

***Example 1: Calculate the Average of Three Numbers***

START

INPUT five numbers and store them in variables num1, num2 and num3

CALCULATE the sum of the three inputted numbers and store them in variable sum

CALCULATE the average of the three inputted numbers and store them in variable avg

PRINT the value of the variable avg

END

**Example 2 Compute Area of a Triangle**

START

Enter the base , B

Enter height, H

Calculate the area = 1/2 * B * H

Display area

END

**Advantages of Pseudocode**

There are several advantages inherent to pseudocode and its use. Some of these include:

- Pseudocode is a simple way to represent an algorithm or program.
- It is written easily in a word processing application and easily modified.
- Pseudocode is easy to understand and can be written by anyone.
- Pseudocode can be used with various structured programming languages.

**Disadvantages of Pseudocode**

The pseudocode also has some disadvantages that should be considered. Some of these disadvantages include:

- Pseudocode is not a programming language, so it cannot be executed by a computer.
- Pseudocode can be ambiguous and sometimes open to interpretation.
- Pseudocode can be difficult to read if not written in a clear and consistent manner.

**Programming Language:**

As we know, to communicate with a person, we need a specific language, similarly to communicate with computers, programmers also need a language is called Programming language.

Language is a mode of communication that is used to share ideas, opinions with each other. For example, if we want to teach someone, we need a language that is understandable by both communicators.

A ***programming language*** is a computer language that is used by programmers/developers to communicate with computers. It is a set of instructions written in any specific language (C, C++, Java, Python) to perform a specific task.

**Types of programming language-**

**1. Low-level programming language**

Low-level language is machine-dependent (0s and 1s) programming language. The processor runs low- level programs directly without the need of a compiler or interpreter, so the programs written in low-level language can be run very fast.

Low-level language is further divided into two parts -

***i. Machine Language***

Machine language is a type of low-level programming language. It is also called as machine code or object code. Machine language is easier to read because it is normally displayed in binary or hexadecimal form (base 16) form. It does not require a translator to convert the programs because computers directly understand the machine language programs.

The advantage of machine language is that it helps the programmer to execute the programs faster than the high-level programming language.

***ii. Assembly Language***

Assembly language (ASM) is also a type of low-level programming language that is designed for specific processors. It represents the set of instructions in a symbolic and human-understandable form. It uses an assembler to convert the assembly language to machine language.

The advantage of assembly language is that it requires less memory and less execution time to execute a program.

**2. High-level programming language**

High-level programming language (HLL) is designed for developing user-friendly software programs and websites. This programming language requires a compiler or interpreter to translate the program into machine language (execute the program).

The main advantage of a high-level language is that it is easy to read, write, and maintain.

High-level programming language includes Python, Java, JavaScript, PHP, C#, C++, C etc.

**Generations of programming language:**

Programming languages have been developed over the year in a phased manner. Each phase of developed has made the programming language more user-friendly, easier to use and more powerful. Each phase of improved made in the development of the programming languages can be referred to as a generation. The programming language in terms of their performance reliability and robustness can be grouped into five **different generations**,

1. First generation languages (1GL)
2. Second generation languages (2GL)
3. Third generation languages (3GL)
4. Fourth generation languages (4GL)
5. Fifth generation languages (5GL)

***1. First Generation Language (Machine language)***

The first generation programming language is also called low-level programming language because they were used to program the computer system at a very low level of abstraction. i.e. at the machine level. The machine language also referred to as the native language of the computer system is the first generation programming language. In the machine language, a programmer only deals with a binary number.

**Advantages of first generation language**

- They are translation free and can be directly executed by the computers.
- The programs written in these languages are executed very speedily and efficiently by the CPU of the computer system.

- The programs written in these languages utilize the memory in an efficient manner because it is possible to keep track of each bit of data.

***2. Second Generation language (Assembly Language)***

The second generation programming language also belongs to the category of low-level-programming language. The second generation language comprises assembly languages that use the concept of mnemonics for the writing program. In the assembly language, symbolic names are used to represent the opcode and the operand part of the instruction.

**Advantages of second generation language**

- It is easy to develop understand and modify the program developed in these languages are compared to those developed in the first generation programming language.
- The programs written in these languages are less prone to errors and therefore can be maintained with a great case.

***3. Third Generation languages (High-Level Languages)***

The third generation programming languages were designed to overcome the various limitations of the first and second generation programming languages. The languages of the third and later generation are considered as a high-level language because they enable the programmer to concentrate only on the logic of the programs without considering the internal architecture of the computer system.

**Advantages of third generation programming language**

- It is easy to develop, learn and understand the program.
- As the program written in these languages are less prone to errors they are easy to maintain.
- The program written in these languages can be developed in very less time as compared to the first and second generation language.

**Examples:** FORTRAN, ALGOL, COBOL, C++, C

***4. Fourth generation language (Very High-level Languages)***

The languages of this generation were considered as very high-level programming languages required a lot of time and effort that affected the productivity of a programmer. The fourth generation programming languages were designed and developed to reduce the time, cost and effort needed to develop different types of software applications.

**Advantages of fourth generation languages**

- These programming languages allow the efficient use of data by implementing the various database.
- They require less time, cost and effort to develop different types of software applications.
- The programs developed in these languages are highly portable as compared to the programs developed in the languages of other generation.

**Examples:** SOL, CSS, coldfusion

***5. Fifth generation language (Artificial Intelligence Language)***

The programming languages of this generation mainly focus on constraint programming. The major fields in which the fifth generation programming language are employed are Artificial Intelligence and Artificial Neural Networks

**Advantages of fifth generation languages**

- These languages can be used to query the database in a fast and efficient manner.
- In this generation of language, the user can communicate with the computer system in a simple and an easy manner.

**Examples:** mercury, prolog, OPS5

**Structured Programming Language:**

In structured programming, we sub-divide the whole program into small modules so that the program becomes easy to understand. The purpose of structured programming is to linearize control flow through a computer program so that the execution sequence follows the sequence in which the code is written. The dynamic structure of the program than resemble the static structure of the program. This enhances the readability, testability, and modifiability of the program. This linear flow of control can be managed by restricting the set of allowed applications construct to a single entry, single exit formats.

We use structured programming because it allows the programmer to understand the program easily. If a program consists of thousands of instructions and an error occurs then it is complicated to find that error in the whole program, but in structured programming, we can easily detect the error and then go to that location and correct it. This saves a lot of time.

**Introduction to C:**

C is a procedural programming language. It was initially developed by *Dennis Ritchie* in the year 1972. It was mainly developed as a system programming language to write an operating system. The main features of the C language include low-level memory access, a simple set of keywords, and a clean style, these features make C language suitable for system programming like an operating system or compiler development.

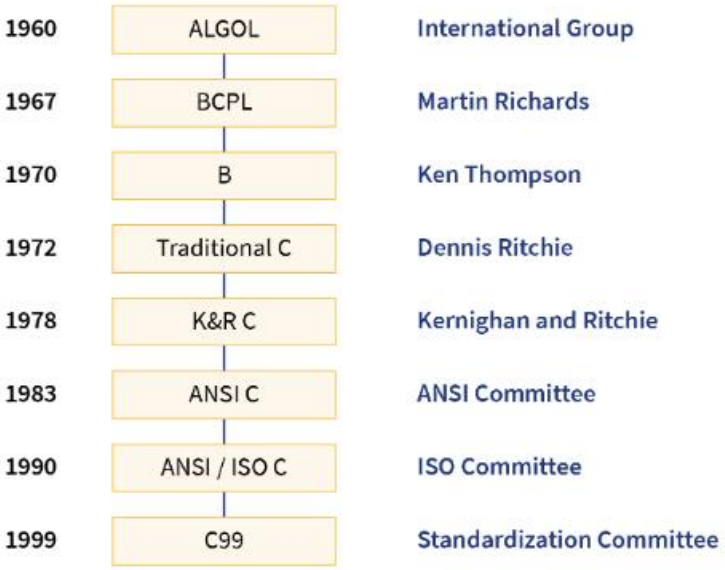
Many later languages have borrowed syntax/features directly or indirectly from the C language. Like syntax of Java, PHP, JavaScript, and many other languages are mainly based on the C language. C++ is nearly a superset of C language.

**C** is a general-purpose programming language that is extremely popular, simple, and flexible to use. It is a structured programming language that is machine.

**History of C Language:**

History of C language is interesting to know. Here we are going to discuss a brief history of the c language. C programming language was developed in 1972 by Dennis Ritchie at bell laboratories of AT&T (American Telephone & Telegraph), located in the U.S.A. Dennis Ritchie is known as the founder of the c language. It was developed to overcome the problems of previous languages such as B, BCPL, etc.

Initially, C language was developed to be used in UNIX operating system. It inherits many features of previous languages such as B and BCPL.




***Features of C Language:***

C is the widely used language. It provides many features that are given below.

***1) Simple***
C is a simple language in the sense that it provides a structured approach (to break the problem into parts), the rich set of library functions, data types, etc.

***2) Machine Independent or Portable***
Unlike assembly language, c programs can be executed on different machines with some machine specific changes. Therefore, C is a machine independent language.

***3) Mid-level programming language***
Although, C is intended to do low-level programming. It is used to develop system applications such as kernel, driver, etc. It also supports the features of a high-level language. That is why it is known as mid-level language.

***4) Structured programming language***
C is a structured programming language in the sense that we can break the program into parts using functions. So, it is easy to understand and modify. Functions also provide code reusability.

***5) Rich Library***
C provides a lot of inbuilt functions that make the development fast.

***6) Dynamic Memory Management***

It supports the feature of dynamic memory allocation. In C language, we can free the allocated memory at any time by calling the free() function.

***7) Speed***

The compilation and execution time of C language is fast since there are lesser inbuilt functions and hence the lesser overhead.

***8) Pointer***

C provides the feature of pointers. We can directly interact with the memory by using the pointers. We can use pointers for memory, structures, functions, array, etc.

***9) Recursion***

In C, we can call the function within the function. It provides code reusability for every function. Recursion enables us to use the approach of backtracking.

***10) Syntax Based Language***

Like most High-level languages, for example, Java, C++, C#, the C language has a syntax, there are proper rules for writing the code, and the C language strictly follows it.If you write anything that is not allowed, you will get a compile-time error, which happens when the compiler is unable to compile your code because of some incorrect code syntax.

***11) Powerful***

C language is a very powerful programming language. It has a broad range of features like support for many data types, operators, keywords, etc., allows structuring of code using functions, loops, decision-making statements, then there are complex data-structures like structures, arrays, etc., and pointers, which makes C quite resourceful and powerful, etc.

***10) Extensible***

C language is extensible because it can easily adopt new features.

| Section | Description |
|---|---|
| ***Documentation*** | Consists of the description of the program, programmer's name, and creation date. These are generally written in the form of comments. |
| ***Link*** | All header files are included in this section which contains different functions from the libraries. A copy of these header files is inserted into your code before compilation. |

**Structure of the C Program:**

| | |
|---|---|
| ***Definition*** | Includes preprocessor directive, which contains symbolic constants. E.g.: #define allows us to use constants in our code. It replaces all the constants with its value in the code. |
| ***Global Declaration*** | Includes declaration of global variables, function declarations, static global variables, and functions. |
| ***Main() Function*** | For every C program, the execution starts from the main() function. It is mandatory to include a main() function in every C program. |
| ***Subprograms*** | Includes all user-defined functions (functions the user provides). They can contain the inbuilt functions and the function definitions declared in the Global Declaration section. These are called in the main() function. |

Example:

```
1.  /* Sum of two numbers */
2.  #include<stdio.h>
3.  int main()
4.  {
5.      int a, b, sum;
6.      printf("Enter two numbers to be added ");
7.      scanf("%d %d", &a, &b);
8.      // calculating sum of two numbers
9.      sum = a + b;
10.     printf("%d + %d = %d", a, b, sum);
11.     return 0;
12. }
```

**C Character Set:**

As every language contains a set of characters used to construct words, statements, etc., C language also has a set of characters which include **alphabets, digits**, and **special symbols**. C language supports a total of 256 characters.

Every C program contains statements. These statements are constructed using words and these words are constructed using characters from C character set. C language character set contains the following set of characters...

1. Alphabets
2. Digits
3. Special Symbols

***Alphabets***

C language supports all the alphabets from the English language. Lower and upper case letters together support 52 alphabets.

lower case letters - **a to z**

UPPER CASE LETTERS - **A to Z**

***Digits***

C language supports 10 digits which are used to construct numerical values in C language.

Digits - **0, 1, 2, 3, 4, 5, 6, 7, 8, 9**

***Special Symbols***

C language supports a rich set of special symbols that include symbols to perform mathematical operations, to check conditions, white spaces, backspaces, and other special symbols.

Special Symbols - **~ @ # $ % ^ & * ( ) _ - + = { } [ ] ; : ' " / ? . > , < etc.**

**Comments in C:**

Comments in C language are used to provide information about lines of code. It is widely used for documenting code. There are 2 types of comments in the C language.

1. Single Line Comments
2. Multi-Line Comments

***Single Line Comments-***

Single line comments are represented by double slash \\. Let's see an example of a single line comment in C.

```
1. #include<stdio.h>
2. int main(){
3. //printing some information
```

```
4. printf("Hello C");
5. return 0;
6. }
```

***Multi Line Comments-***

Multi-Line comments are represented by slash asterisk * ... *\. It can occupy many lines of code, but it can't be nested. Syntax:

```
1. #include<stdio.h>
2. int main(){
3. /*printing some information
4. Multi-Line Comment*/
5. printf("Hello C");
6. return 0;
7. }
```

**Tokens in C:**

Tokens in C are the most important element to be used in creating a program in C. We can define the token as the smallest individual element in C. For `example, you cannot create a sentence without using words; similarly, we cannot create a program in C without using tokens in C. Therefore, tokens in C is the building block or the basic component for creating a program in C language.

***Classification of tokens in C***

Tokens in C language can be divided into the following categories:

- Keywords
- Identifiers
- Strings

- Operators
- Constant
- Special Characters

**1. Keywords:** Keywords are pre-defined or reserved words in a programming language. Each keyword is meant to perform a specific function in a program. Since keywords are referred names for a compiler, they can't be used as variable names because by doing so, we are trying to assign a new meaning to the keyword which is not allowed. You cannot redefine keywords. However, you can specify the text to be substituted for keywords before compilation by using C/C++ preprocessor directives. C language supports 32 keywords which are given below:

| | | | |
|---|---|---|---|
| auto | double | int | struct |
| break | else | long | switch |
| case | enum | register | typedef |
| char | extern | return | union |
| const | float | short | unsigned |
| continue | for | signed | void |
| default | goto | sizeof | volatile |
| do | if | static | while |

**2. Identifiers:** Identifiers are used as the general terminology for the naming of variables, functions and arrays. These are user-defined names consisting of an arbitrarily long sequence of letters and digits with either a letter or the underscore(_) as a first character. Identifier names must differ in spelling and case from any keywords. You cannot use keywords as identifiers; they are reserved for special use. Once declared, you can use the identifier in later program statements to refer to the associated value.

***There are certain rules that should be followed while naming c identifiers:***

- They must begin with a letter or underscore(_).
- They must consist of only letters, digits, or underscore. No other special character is allowed.
- It should not be a keyword.
- It must not contain white space.
- It should be up to 31 characters long as only the first 31 characters are significant.

**3. Strings**: Strings are nothing but an array of characters ended with a null character ('\0'). This null character indicates the end of the string. Strings are always enclosed in double-quotes. Whereas, a character is enclosed in single quotes in C. Declarations for String:

char string[20] = {'p', 'r', 'o', 'g', 'r', 'a', 'm', 'm', 'i', 'n', 'g', '\0'};
char string[20] = "programming";
char string [] = "programming";
when we declare char as "string[20]", 20 bytes of memory space is allocated for holding the string value.
When we declare char as "string[]", memory space will be allocated as per the requirement during the execution of the program.

**4. Operators:** Operators are symbols that trigger an action when applied to C variables and other objects. The data items on which operators act upon are called operands.
Depending on the number of operands that an operator can act upon, operators can be classified as follows:

- ***Unary Operators:*** Those operators that require only a single operand to act upon are known as unary operators. For Example increment and decrement operators
- ***Binary Operators:*** Those operators that require two operands to act upon are called binary operators. Binary operators are classified into :

  Arithmetic operators
  Relational Operators
  Logical Operators
  Assignment Operators
  Bitwise Operator

- ***Ternary Operator:*** The operator that require three operands to act upon are called ternary operator. Conditional Operator(?) is also called ternary operator.

  Syntax: (Expression1)? expression2: expression3;

**5. Constants:** A constant is a value assigned to the variable which will remain the same throughout the program, i.e., the constant value cannot be changed.
There are two ways of declaring constant:

Using const keyword
Using #define pre-processor

**6. Special characters:** Some special characters are used in C, and they have a special meaning which cannot be used for another purpose.

- **Square brackets [ ]:** The opening and closing brackets represent the single and multidimensional subscripts.
- **Simple brackets ( ):** It is used in function declaration and function calling. For example, printf() is a pre-defined function.
- **Curly braces { }:** It is used in the opening and closing of the code. It is used in the opening and closing of the loops.
- **Comma (,):** It is used for separating for more than one statement and for example, separating function parameters in a function call, separating the variable when printing the value of more than one variable using a single printf statement.
- **Hash/pre-processor (#):** It is used for pre-processor directive. It basically denotes that we are using the header file.
- **Asterisk (*):** This symbol is used to represent pointers and also used as an operator for multiplication.
- **Tilde (~):** It is used as a destructor to free memory.
- **Period (.):** It is used to access a member of a structure or a union.

**DataType:**

Data types in C are specified or identified as the data storage format that tells the compiler or interpreter how the programmer enters the data and what type of data they enter into the program. Data types are used to define a variable before use in a program. Data types determine the size of the variable, space it occupies in storage.

C language supports a wide variety of data types to accommodate any data manipulation. The variety of data types available allows the programmer to select the type appropriate to the program's needs and the machine.

***Basic types***

| Type | Size (bytes) | Format Specifier |
|---|---|---|
| int | at least 2, usually 4 | %d, %i |
| char | 1 | %c |
| float | 4 | %f |
| double | 8 | %lf |
| short int | 2 usually | %hd |
| unsigned int | at least 2, usually 4 | %u |
| long int | at least 4, usually 8 | %ld, %li |
| long long int | at least 8 | %lld, %lli |
| unsigned long int | at least 4 | %lu |
| unsigned long long int | at least 8 | %llu |
| signed char | 1 | %c |
| unsigned char | 1 | %c |
| long double | at least 10, usually 12 or 16 | %Lf |

***Derived Data Types:***
A derived type formed by using the basic types in combination. Array, function and pointers are the derived types. Here, we are discussing them briefly.

***Array-***
An array is a collection of elements of the same data type. It used to handle a large amount of data, without the need to declare many individual variables separately. The array elements stored in contiguous memory locations (i.e. one after the other). All the array elements must either any primary data type like int, float, char, double etc., or they can be any user-defined data type like structure and unions.

***Functions-***
A function is a self-contained block of statements that performs a specific task. It allows us to avoid duplicating code that used more than once. Using functions, in the extensive program, can

divide into smaller self-contained parts that are easier for us and others to understand, modify and maintain.
Functions can divided into two categories: library functions (in-built functions) and user-defined functions.

***Pointer-***
A pointer is a variable that contains the address of the data items such as variable or function or array rather than a value. It is a derived data type as it built from one of the basic types available in C.

***User-Defined Types:***
As the name suggests, these data types are created by users using one or more basic types in combination, and other derived and user-defined types.
Structure, union, enum type definitions help to define user-defined types.

***Structure-***
A structure is a collection of related data items which can be of different types, having a single UNIT name.

***Union-***
Unions consist of one or more members which may be of different data types just like structures. However, unlike structures where each member is assigned a unique storage area, in the union, all the members share the same storage area within the computer's memory.

***Enumerated Data Types-***
C provides a special kind of user-defined data type known as enumerated type explicitly designed for variables that can take a small set of possible values.

**Input and output in C:**
When we say Input, it means to feed some data into a program. An input can be given in the form of a file or from the command line. C programming provides a set of built-in functions to read the given input and feed it to the program as per requirement.

When we say Output, it means to display some data on screen, printer, or in any file. C programming provides a set of built-in functions to output the data on the computer screen as well as to save it in text or binary files.

The getchar() and putchar() Functions

The int getchar(void) function reads the next available character from the screen and returns it as an integer. This function reads only single character at a time. You can use this method in the loop in case you want to read more than one character from the screen.

The int putchar(int c) function puts the passed character on the screen and returns the same character. This function puts only single character at a time. You can use this method in the loop in case you want to display more than one character on the screen.

Example –

```
#include <stdio.h>
void main( ) {
   int c;
   printf("Enter a value :");
   c = getchar( );

   printf( "Entered: ");
   putchar( c );
}
```

**The gets() and puts() Functions**

The char *gets(char *s) function reads a line from stdin into the buffer pointed to by s until either a terminating newline or EOF (End of File).

The int puts(const char *s) function writes the string 's' and 'a' trailing newline to stdout.

```
#include <stdio.h>
int main( ) {

   char str[100];

   printf( "Enter a value :");
   gets( str );

   printf( "\nYou entered: ");
   puts( str );

   return 0;
}
```

**The scanf() and printf() Functions**

The int scanf(const char *format, ...) function reads the input from the standard input stream stdin and scans that input according to the format provided.

The int printf(const char *format, ...) function writes the output to the standard output stream stdout and produces the output according to the format provided.

The format can be a simple constant string, but you can specify %s, %d, %c, %f, etc., to print or read strings, integer, character or float respectively. There are many other formatting options available which can be used based on requirements.

```
#include <stdio.h>
void main( ) {

  char str[100];
  int i;

  printf( "Enter a value :");
  scanf("%s %d", str, &i);

  printf( "You entered: %s %d ", str, i);

}
```

**Escape Sequence in C:**

An escape sequence in C language is a sequence of characters that doesn't represent itself when used inside string literal or character.

It is composed of two or more characters starting with backslash \. For example: \n represents new line.

| Escape Sequence | Purpose |
|---|---|
| \a | Alarm or Beep |
| \b | Backspace |
| \f | Form Feed |
| \n | New Line |
| \r | Carriage Return |
| \t | Tab (Horizontal) |

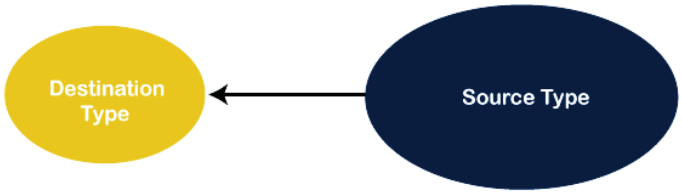
| | |
|---|---|
| \v | Vertical Tab |
| \\ | Backslash |
| \' | Single Quote |
| \" | Double Quote |
| \? | Question Mark |
| \nnn | octal number |
| \xhh | hexadecimal number |
| \0 | Null |

**Type Conversion and Type Casting:**

The two terms type casting and the type conversion are used in a program to convert one data type to another data type. The conversion of data type is possible only by the compiler when they are compatible with each other. Let's discuss the difference between type casting and type conversion in any programming language.

***What is a type casting?***

When a data type is converted into another data type by a programmer or user while writing a program code of any programming language, the mechanism is known as type casting. The programmer manually uses it to convert one data type into another. It is used if we want to change the target data type to another data type. Remember that the destination data type must be smaller than the source data type. Hence it is also called a narrowing conversion.




**Syntax:**

**Destination_datatype = (target_datatype) variable;**

(data_type) it is known as casting operator

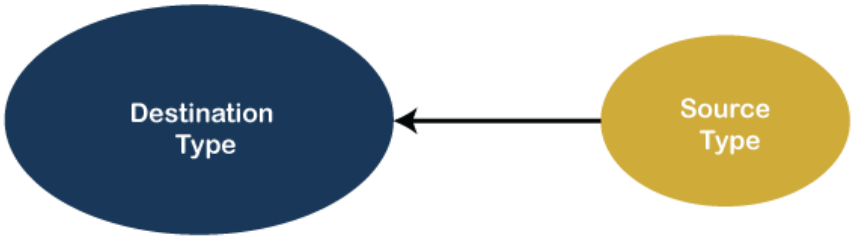
***Target_datatype:*** It is the data type in which we want to convert the destination data type. The variable defines a value that is to be converted in the target_data type. Let's understand the concept of type casting with an example.

Suppose, we want to convert the float data type into int data type. Here, the target data type is smaller than the source data because the size of int is 2 bytes, and the size of the float data type is

4 bytes. And when we change it, the value of the float variable is truncated and convert into an integer variable. Casting can be done with a compatible and non-compatible data type.

***float b = 3.0;***
***int a = (int) b; // converting a float value into integer***

***What is type conversion?***
If a data type is automatically converted into another data type at compile time is known as type conversion. The conversion is performed by the compiler if both data types are compatible with each other. Remember that the destination data type should not be smaller than the source type. It is also known as widening conversion of the data type.




Let's understand the type conversion with an example.

Suppose, we have an int data type and want to convert it into a float data type. These are data types compatible with each other because their types are numeric, and the size of int is 2 bytes which is smaller than float data type. Hence, the compiler automatically converts the data types without losing or truncating the values.

**int a = 20;**
**float b;**
**b = a; // Now the value of variable b is 20.000**

/* It defines the conversion of int data type to float data type without losing the information. */

## C Operators & Expressions:

Operators are the foundation of any programming language. We can define operators as symbols that help us to perform specific mathematical and logical computations on operands. In other words, we can say that an operator operates the operands.

An operator is a symbol that operates on a value or a variable. For example: + is an operator to perform addition. C has a wide range of operators to perform various operations.

### *C Arithmetic Operators-*

An arithmetic operator performs mathematical operations such as addition, subtraction, multiplication, division etc on numerical values (constants and variables).

| ***Operator*** | ***Meaning of Operator*** |
|---|---|
| + | addition or unary plus |
| - | subtraction or unary minus |
| * | multiplication |
| / | division |
| % | remainder after division (modulo division) |

### *C Increment and Decrement Operators-*

C programming has two operators increment ++ and decrement -- to change the value of an operand (constant or variable) by 1.

Increment ++ increases the value by 1 whereas decrement -- decreases the value by 1. These two operators are unary operators, meaning they only operate on a single operand.

### *C Assignment Operators-*

An assignment operator is used for assigning a value to a variable. The most common assignment operator is =

| ***Operator*** | ***Example*** | ***Same as*** |
|---|---|---|
| = | a = b | a = b |
| += | a += b | a = a+b |
| -= | a -= b | a = a-b |
| *= | a *= b | a = a*b |
| /= | a /= b | a = a/b |
| %= | a %= b | a = a%b |

***C Relational Operators-***

A relational operator checks the relationship between two operands. If the relation is true, it returns 1; if the relation is false, it returns value 0.

Relational operators are used in decision making and loops.

| ***Operator*** | ***Meaning of Operator*** | ***Example*** |
|---|---|---|
| == | Equal to | 5 == 3 is evaluated to 0 |
| > | Greater than | 5 > 3 is evaluated to 1 |
| < | Less than | 5 < 3 is evaluated to 0 |
| != | Not equal to | 5 != 3 is evaluated to 1 |
| >= | Greater than or equal to | 5 >= 3 is evaluated to 1 |
| <= | Less than or equal to | 5 <= 3 is evaluated to 0 |

***C Logical Operators-***

An expression containing logical operator returns either 0 or 1 depending upon whether expression results true or false. Logical operators are commonly used in decision making in C programming.

| | | |
|---|---|---|
| && | Logical AND. | True only if all operands are true |
| \|\| | Logical OR. | True only if either one operand is true |
| ! | Logical NOT. | True only if the operand is 0 |

***C Bitwise Operators-***

During computation, mathematical operations like: addition, subtraction, multiplication, division, etc are converted to bit-level which makes processing faster and saves power.

Bitwise operators are used in C programming to perform bit-level operations.

| ***Operators*** | ***Meaning of operators*** |
|---|---|
| & | Bitwise AND |
| \| | Bitwise OR |
| ^ | Bitwise exclusive OR |
| ~ | Bitwise complement |
| << | Shift left |
| >> | Shift right |

***Conditional Operator-***

The operator pair "?" and ":" is known as conditional operator. These pair of operators are ternary operators. The general syntax of conditional operator is:

***expression1 ? expression2 : expression3 ;***

x = (a > b) ? a : b ;

**Operator Precedence and Associatively:**

***Precedence of operator-***

The precedence of operators determines which operator is executed first if there is more than one operator in an expression.

Let us consider an example:

**int x = 5 - 17* 6;**

In C, the precedence of * is higher than - and =. Hence, 17 * 6 is evaluated first. Then the expression involving - is evaluated as the precedence of - is higher than that of =.

Here's a table of operators precedence from higher to lower.

The associativity of operators determines the direction in which an expression is evaluated.

| Category | Operator | Associativity |
|---|---|---|
| Postfix | () [] -> . ++ - - | Left to right |
| Unary | + - ! ~ ++ - - (type)* & sizeof | Right to left |
| Multiplicative | * / % | Left to right |
| Additive | + - | Left to right |
| Shift | << >> | Left to right |
| Relational | < <= > >= | Left to right |
| Equality | == != | Left to right |
| Bitwise AND | & | Left to right |
| Bitwise XOR | ^ | Left to right |
| Bitwise OR | \| | Left to right |
| Logical AND | && | Left to right |
| Logical OR | \|\| | Left to right |
| Conditional | ?: | Right to left |
| Assignment | = += -= *= /= %=>>= <<= &= ^= \|= | Right to left |
| Comma | , | Left to right |

**Associativity of Operators-**

The associativity of operators determines the direction in which an expression is evaluated. For example,

**b = a;**

Here, the value of a is assigned to b, and not the other way around. It's because the associativity of the = operator is from right to left.

Also, if two operators of the same precedence (priority) are present, associativity determines the direction in which they execute.

Let us consider an example:

**1 == 2 != 3**

Here, operators == and != have the same precedence. And, their associativity is from left to right. Hence, 1 == 2 is executed first.

**Control Structures / Statements in C Language:**

A program is nothing but the execution of sequence of one or more instructions.

Quite often, it is desirable to alter the sequence of the statements in the program depending upon certain circumstances. (i.e., we have a number of situations where we may have to change the order of execution of statements based on certain conditions)

(Or)

Repeat a group of statements until certain specified conditions are met.

This involves a kind of decision making to see whether a particular condition has occurred or not and direct the computer to execute certain statements accordingly.

Based on application, it is necessary / essential

(i) To alter the flow of a program

(ii) Test the logical conditions

(iii) Control the flow of execution as per the selection these conditions can be placed in the program using decision-making statements.

C Language supports mainly three types of control statements-

***1. Decision making statements***
a) Simple if Statement
b) if – else Statement
c) Nested if-else statement
d) else – if Ladder
e) switch statement

***2. Loop control statements***
a) for Loop
b) while Loop
c) do-while Loop

***3. Unconditional control statements***
a) goto Statement
b) break Statement
c) continue Statement
d) return statement

***1) Simple "if" statement :***
"if" statement is a powerful decision making statement, and is used to control the flow of execution of statements.

***Syntax :***
if (Condition or test expression)
Statement;
Rest of the program

(OR)

if (Condition or test expression)
{
Statement(s);
}
Rest of the program;

***(2) "if-else" Statement:***

It is observed that the "if" statement executes only when the condition following if is true. It does nothing when the condition is false.

In if-else either True-Block or False – Block will be executed and not both. The "else" Statement cannot be used without "if".

**Syntax :**

```
if ( Test Expression or Condition )
    {
      Statements;  /*true block (or) if block */
     }
    else
     {
      Statements;   /* false block (or) else block */
     }
```

***(3) nested-if statement:***

A nested if in C is an if statement that is the target of another if statement. Nested if statements mean an if statement inside another if statement. Yes, both C and C++ allow us to nested if statements within if statements, i.e, we can place an if statement inside another if statement.

Syntax:

```
if (condition1)
{
  // Executes when condition1 is true
  if (condition2)
  {
    // Executes when condition2 is true
  }
}
```

***(4) if-else-if ladder:***

The C if statements are executed from the top down. As soon as one of the conditions controlling the if is true, the statement associated with that if is executed, and the rest of the C else-if ladder is bypassed. If none of the conditions is true, then the final else statement will be executed.

Syntax:

```
if (condition)
   statement;
else if (condition)
   statement;
.
.

else
   statement;
```

**switch Statement:**
The switch statement allows us to execute one code block among many alternatives.

We can do the same thing with the if...else..if ladder. However, the syntax of the switch statement is much easier to read and write.

**Syntax of switch...case**

```
switch (expression)
{
   case constant1:
     // statements
     break;

   case constant2:
     // statements
     break;
   .
   .
   .
   default:
     // default statements
}
```

***How does the switch statement work?***

The expression is evaluated once and compared with the values of each case label.

If there is a match, the corresponding statements after the matching label are executed. For example, if the value of the expression is equal to constant2, statements after case constant2: are executed until break is encountered.
If there is no match, the default statements are executed.

**Iterative Statements (Loop):**
The statements that cause a set of statements to be executed repeatedly either for a specific number of times or until some condition is satisfied are known as iteration statements. That is, as long as the condition evaluates to True, the set of statement(s) is executed. The various iteration statements used in C++ are for loop, while loop and do while loop.

***The for Loop***

The for loop is one of the most widely used loops in C++. The for loop is a deterministic loop in nature, that is, the number of times the body of the loop is executed is known in advance.

The syntax of the for loop is

```
for(initialize; condition; update) {
  //body of the for loop
}
```

***The while Loop***

The while loop is used to perform looping operations in situations where the number of iterations is not known in advance. That is, unlike the for loop, the while loop is non deterministic in nature.

The syntax of the while loop is

```
while(condition) {
  // body of while loop
}
```
These points should be noted about the while loop.

***The do-while loop***
In a while loop, the condition is evaluated at the beginning of the loop and if the condition evaluates to False, the body of the loop is not executed even once. However, if the body of the loop is to be executed at least once, no matter whether the initial state of the condition is True or

False, the do-while loop is used. This loop places the condition to be evaluated at the end of the loop.

The syntax of the do-while loops is given here.

```
do {
  //body of do while loop
}while(condition) ;
```

**Jump Statements in C:**

These statements are used in C or C++ for the unconditional flow of control throughout the functions in a program. They support four types of jump statements:

**A) break**

This loop control statement is used to terminate the loop. As soon as the break statement is encountered from within a loop, the loop iterations stop there, and control returns from the loop immediately to the first statement after the loop.

Syntax:
break;

**B) continue**

This loop control statement is just like the break statement. The continue statement is opposite to that of the break statement, instead of terminating the loop, it forces to execute the next iteration of the loop.

As the name suggests the continue statement forces the loop to continue or execute the next iteration. When the continue statement is executed in the loop, the code inside the loop following the continue statement will be skipped and the next iteration of the loop will begin.

Syntax:
continue;

**C) goto**

The goto statement in C/C++ also referred to as the unconditional jump statement can be used to jump from one point to another within a function.

Syntax:

goto label;

goto label;
…….
label: statement;

**D) return**

The return in C or C++ returns the flow of the execution to the function from where it is called. This statement does not mandatorily need any conditional statements. As soon as the statement is executed, the flow of the program stops immediately and returns the control from where it was called. The return statement may or may not return anything for a void function, but for a non-void function, a return value must be returned.

Syntax:

return[expression];

**C Functions:**

In c, we can divide a large program into the basic building blocks known as function. The function contains the set of programming statements enclosed by {}. A function can be called multiple times to provide reusability and modularity to the C program. In other words, we can say that the collection of functions creates a program. The function is also known as procedure or subroutine in other programming languages.

***Advantage of functions in C***

There are the following advantages of C functions.

- By using functions, we can avoid rewriting same logic/code again and again in a program.
- We can call C functions any number of times in a program and from any place in a program.
- We can track a large C program easily when it is divided into multiple functions.
- Reusability is the main achievement of C functions.
- However, Function calling is always a overhead in a C program.

***Types of Functions***

There are two types of functions in C programming:

***Library Functions:*** are the functions which are declared in the C header files such as scanf(), printf(), gets(), puts(), ceil(), floor() etc.

***User-defined functions:*** are the functions which are created by the C programmer, so that he/she can use it many times. It reduces the complexity of a big program and optimizes the code.

**Storage Classes in C:**

A storage class represents the visibility and a location of a variable. It tells from what part of code we can access a variable. A storage class in C is used to describe the following things:

- The variable scope.
- The location where the variable will be stored.
- The initialized value of a variable.
- A lifetime of a variable.
- Who can access a variable?

***There are four types of storage classes in C***

1. Automatic
2. External
3. Static
4. Register

| Storage Classes | Storage Place | Default Value | Scope | Lifetime |
|---|---|---|---|---|
| **auto** | RAM | Garbage Value | Local | Within function |
| **extern** | RAM | Zero | Global | Till the end of the main program Maybe declared anywhere in the program |
| **static** | RAM | Zero | Local | Till the end of the main program, Retains value between multiple functions call |
| **register** | Register | Garbage Value | Local | Within the function |

**Recursion / Recursive Function:**

Recursion is the technique of making a function call itself. This technique provides a way to break complicated problems down into simple problems which are easier to solve.

```
void recurse()
{
   ... .. ...
   recurse();
   ... .. ...
}

int main()
{
   ... .. ...
   recurse();
   ... .. ...
}
```

***Different types of the recursion-***

1. Direct Recursion
2. Indirect Recursion
3. Tail Recursion
4. No Tail/ Head Recursion
5. Linear recursion
6. Tree Recursion

**Array:**

An array is defined as the collection of similar type of data items stored at contiguous memory locations. Arrays are the derived data type in C programming language which can store the primitive type of data such as int, char, double, float, etc. It also has the capability to store the collection of derived data types, such as pointers, structure, etc. The array is the simplest data structure where each data element can be randomly accessed by using its index number.

C array is beneficial if you have to store similar elements. For example, if we want to store the marks of a student in 6 subjects, then we don't need to define different variables for the marks in the different subject. Instead of that, we can define an array which can store the marks in each subject at the contiguous memory locations.

By using the array, we can access the elements easily. Only a few lines of code are required to access the elements of the array.

***Properties of Array***

The array contains the following properties.
Each element of an array is of same data type and carries the same size, i.e., int = 4 bytes.
Elements of the array are stored at contiguous memory locations where the first element is stored at the smallest memory location.
Elements of the array can be randomly accessed since we can calculate the address of each element of the array with the given base address and the size of the data element.

***Advantage of C Array***

1) Code Optimization: Less code to the access the data.
2) Ease of traversing: By using the for loop, we can retrieve the elements of an array easily.
3) Ease of sorting: To sort the elements of the array, we need a few lines of code only.
4) Random Access: We can access any element randomly using the array.

***Disadvantage of C Array***

Fixed Size: Whatever size, we define at the time of declaration of the array, we can't exceed the limit. So, it doesn't grow the size dynamically like LinkedList.

**Declaration of C Array:**

We can declare an array in the c language in the following way.

data_type array_name[array_size];
Example to declare the array-
int marks[5];

**Initialization of C Array:**
The simplest way to initialize an array is by using the index of each element. We can initialize each element of the array by using the index. Consider the following example.

```
marks[0]=10;//initialization of array
marks[1]=20;
marks[2]=30;
marks[3]=40;
marks[4]=50;
```

***C Array: Declaration with Initialization***
We can initialize the c array at the time of declaration. Let's see the code.

```
int marks[5]={20,30,40,50,60};
```
In such case, there is no requirement to define the size. So it may also be written as the following code.
```
int marks[]={20,30,40,50,60};
```

**Two Dimensional Array in C:**
The two-dimensional array can be defined as an array of arrays. The 2D array is organized as matrices which can be represented as the collection of rows and columns. However, 2D arrays are created to implement a relational database lookalike data structure. It provides ease of holding the bulk of data at once which can be passed to any number of functions wherever required.

***Declaration of two dimensional Array in C***
The syntax to declare the 2D array is given below.

```
data_type array_name[rows][columns];
```
Example:
```
int twodimen[4][3];
```

***Initialization of 2D Array in C***
In the 1D array, we don't need to specify the size of the array if the declaration and initialization are being done simultaneously. However, this will not work with 2D arrays. We will have to define at least the second dimension of the array. The two-dimensional array can be declared and defined in the following way.

int arr[4][3]={{1,2,3},{2,3,4},{3,4,5},{4,5,6}};

**String:**

In C programming, a string is a sequence of characters terminated with a null character '\0'. Strings are defined as an array of characters. The difference between a character array and a string is the string is terminated with a unique character '\0'.

***Declaration of Strings***

Declaring a string is as simple as declaring a one-dimensional array. Below is the basic syntax for declaring a string.

char str_name[size];

***Initializing a String***

A string can be initialized in different ways. We will explain this with the help of an example. Below are the examples to declare a string with the name str and initialize it with "Welcome".

***4 Ways to Initialize a String in C-***

1. Assigning a string literal without size: String literals can be assigned without size. Here, the name of the string str acts as a pointer because it is an array.

char str[] = "Welcome";

2. Assigning a string literal with a predefined size: String literals can be assigned with a predefined size. But we should always account for one extra space which will be assigned to the null character. If we want to store a string of size n then we should always declare a string with a size equal to or greater than n+1.

char str[50] = "Welcome";

3. Assigning character by character with size: We can also assign a string character by character. But we should remember to set the end character as '\0' which is a null character.

char str[20] = { 'W','e','l','c','o','m','e','\0'};

4. Assigning character by character without size: We can assign character by character without size with the NULL character at the end. The size of the string is determined by the compiler automatically.

```
char str[] = { 'W','e','l','c','o','m','e','\0'};
```

**String Functions:**

There are many important string functions defined in "string.h" library.

| No. | Function | Description |
|---|---|---|
| 1) | strlen() | returns the length of string name. |
| 2) | strcpy() | copies the contents of source string to destination string. |
| 3) | strcat() | concats or joins first string with second string. The result of the string is stored in first string. |
| 4) | strcmp() | compares the first string with second string. If both strings are same, it returns 0. |
| 5) | strrev() | returns reverse string. |
| 6) | strlwr() | returns string characters in lowercase. |
| 7) | strupr() | returns string characters in uppercase. |

**Character handling functions:**

All data is entered into computers as characters, which includes letters, digits and various special symbols. In this section, we discuss the capabilities of C++ for examining and manipulating individual characters.

The character-handling library includes several functions that perform useful tests and manipulations of character data. Each function receives a character, represented as an int, or EOF as an argument. Characters are often manipulated as integers.

Remember that EOF normally has the value –1 and that some hardware architectures do not allow negative values to be stored in char variables. Therefore, the character-handling functions manipulate characters as integers.

| S.No. | Prototype & Description |
|---|---|
| 1 | **int isdigit( int c )**<br>Returns 1 if c is a digit and 0 otherwise. |
| 2 | **int isalpha( int c )**<br>Returns 1 if c is a letter and 0 otherwise. |
| 3 | **int isalnum( int c )**<br>Returns 1 if c is a digit or a letter and 0 otherwise. |
| 4 | **int isxdigit( int c )**<br>Returns 1 if c is a hexadecimal digit character and 0 otherwise.<br>(See Appendix D, Number Systems, for a detailed explanation of binary, octal, decimal and hexadecimal numbers.) |
| 5 | **int islower( int c )**<br>Returns 1 if c is a lowercase letter and 0 otherwise. |
| 6 | **int isupper( int c )**<br>Returns 1 if c is an uppercase letter; 0 otherwise. |
| 7 | **int isspace( int c )**<br>Returns 1 if c is a white-space character—newline ('\n'), space (' '), form feed ('\f'), carriage return ('\r'), horizontal tab ('\t'), or vertical tab ('\v')—and 0 otherwise. |
| 8 | **int iscntrl( int c )**<br>Returns 1 if c is a control character, such as newline ('\n'), form feed ('\f'), carriage return ('\r'), horizontal tab ('\t'), vertical tab ('\v'), alert ('\a'), or backspace ('\b')—and 0 otherwise. |
| 9 | **int ispunct( int c )**<br>Returns 1 if c is a printing character other than a space, a digit, or a letter and 0 otherwise. |
| 10 | **int isprint( int c )**<br>Returns 1 if c is a printing character including space (' ') and 0 otherwise. |
| 11 | **int isgraph( int c )**<br>Returns 1 if c is a printing character other than space (' ') and 0 otherwise. |

**Pointers:**

Pointers in C are used to store the address of variables or a memory location. This variable can be of any data type i.e, int, char, function, array, or any other pointer. The pointer of type void is called Void pointer or Generic pointer. Void pointer can hold address of any type of variable. The size of the pointer depends on the computer architecture like 16-bit, 32-bit, and 64-bit.

**Syntax**: datatype *var_name;

**Example**: int *ptr;

**Program**:

```
void main()
{       int var = 20;
        int* ptr;
        ptr = &var;
        printf("Value at ptr = %p \n", ptr);
        printf("Value at var = %d \n", var);
        printf("Value at *ptr = %d \n", *ptr);
}
```

To declare a pointer variable: When a pointer variable is declared in C, there must be a * before its name.

**Advantages of Pointers**

- Pointers are used for dynamic memory allocation and deallocation.
- An Array or structure can be accessed efficiently with pointers
- Pointers are useful for accessing memory locations.
- Pointers are used to form complex data structures such as linked lists, graphs, trees, etc.
- Pointers reduce length of the program and its execution time as well.

**Disadvantages of pointers**

- Memory corruption can occur if an incorrect value is provided to pointers
- Pointers are Complex to understand.
- Pointers are majorly responsible for memory leaks.
- Pointers are comparatively slower than variables in C.
- Uninitialized pointers might cause segmentation fault.

**Void Pointers -**

A Void Pointer in C can be defined as an address of any variable. It has no standard data type. A void pointer is created by using the keyword void.

A void pointer is a pointer that has no associated data type with it. A void pointer can hold address of any type and can be typecasted to any type.

```
int main()
{
        int a = 10;
        char b = 'x';

        void* p = &a; // void pointer holds address of int 'a'
        p = &b; // void pointer holds address of char 'b'
}
```

**Pointer Arithmetic in C**

We can perform arithmetic operations on the pointers like addition, subtraction, etc. However, as we know that pointer contains the address, the result of an arithmetic operation performed on the pointer will also be a pointer if the other operand is of type integer. In pointer-from-pointer subtraction, the result will be an integer value. Following arithmetic operations are possible on the pointer in C language:

- Increment
- Decrement
- Addition
- Subtraction
- Comparison

**Incrementing Pointer -**

If we increment a pointer by 1, the pointer will start pointing to the immediate next location. This is somewhat different from the general arithmetic since the value of the pointer will get increased by the size of the data type to which the pointer is pointing.

We can traverse an array by using the increment operation on a pointer which will keep pointing to every element of the array, perform some operation on that, and update itself in a loop.

```
void main()
{
int number=20;
int *p;
p=&number;//stores the address of number variable
printf("Address of p variable is %u \n",p);
p=p+1;
printf("After increment: Address of p variable is %u \n",p);
}
```

***Traversing an array by using pointer***

```
void main ()
{
   int a[5] = {10, 20, 30, 40, 50};
   int *p = a;
   int i;
   printf("Array elements...\n");
   for(i = 0; i< 5; i++)
   {   printf("%d  ",*(p+i));
   }
}
```

**Passing Pointer to a Function in C Programming:**

Just like any other argument, pointers can also be passed to a function as an argument. When we pass a pointer as an argument instead of a variable then the address of the variable is passed instead of the value. So any change made by the function using the pointer is permanently made at the address of passed variable. This technique is known as call by reference in C.

**Example 1:**

```
void salary_slip(int  *var, int b)
{
   *var = *var+b;
}
void main()
{
```

```
   int salary=0, bonus=0;
   printf("Enter the employee current salary:");
   scanf("%d", &salary);
   printf("Enter bonus:");
   scanf("%d", &bonus);
   salary_slip(&salary, bonus);
   printf("Final salary: %d", salary);
}
```

**Example 2:**

```
#include <stdio.h>
void swap(int *n1, int *n2);

void main()
{
   int num1 = 5, num2 = 10;
   swap( &num1, &num2);
   printf("num1 = %d\n", num1);
   printf("num2 = %d", num2);
}

void swap(int* n1, int* n2)
{
   int temp;
   temp = *n1;
   *n1 = *n2;
   *n2 = temp;
}
```

**Passing Array to Function–**

If you want to pass a single-dimension array as an argument in a function, you would have to declare a formal parameter in one of following three ways and all three declaration methods produce similar results because each tells the compiler that an integer pointer is going to be received. Similarly, you can pass multi-dimensional arrays as formal parameters.

```
double getAverage(int arr[], int size);

void main ()
{
   int balance[5] = {1000, 2, 3, 17, 50};
   double avg;
   avg = getAverage( balance, 5 ) ;
   printf( "Average value is: %f ", avg );
 }


double getAverage(int arr[], int size)
{
   int i;
   double avg;
   double sum = 0;

   for (i = 0; i < size; ++i) {
      sum += arr[i];
   }
   avg = sum / size;
   return avg;
}
```

**Memory Allocation:**

The concept of dynamic memory allocation in c language enables the C programmer to allocate memory at runtime. Dynamic memory allocation in c language is possible by 4 functions of stdlib.h/malloc.h header file.

| | |
|---|---|
| malloc() | allocates single block of requested memory. |
| calloc() | allocates multiple block of requested memory. |
| realloc() | reallocates the memory occupied by malloc() or calloc() functions. |
| free() | frees the dynamically allocated memory. |

```
#include<stdio.h>
#include<stdlib.h>
#include<mallloc.h>

int main(){
  int n,i,*ptr,sum=0;
    printf("Enter number of elements: ");
    scanf("%d",&n);
    ptr=(int*)malloc(n*sizeof(int));
    if(ptr==NULL)
    {
        printf("Sorry! unable to allocate memory");
        exit(0);
    }
    printf("Enter elements of array: ");
    for(i=0;i<n;++i)
    {
        scanf("%d",ptr+i);
        sum+=*(ptr+i);
    }
    printf("Sum=%d",sum);
    free(ptr);
return 0;
}
```

**Structure:**

In C, there are cases where we need to store multiple attributes of an entity. It is not necessary that an entity has all the information of one type only. It can have different attributes of different data types. For example, an entity Student may have its name (string), roll number (int), marks (float). To store such type of information regarding an entity student, we have the following approaches:

1. Construct individual arrays for storing names, roll numbers, and marks.
2. Use a special data structure to store the collection of different data types.

***Syntax:***

```
struct structureName {
  dataType member1;
  dataType member2;
  ...
};
```

***Example:***

```
struct Person {
  char name[50];
  int citNo;
  float salary;
};
```

**Program:**

```
struct Person {
  char name[50];
  int citNo;
  float salary;
} person1;

void main() {

  strcpy(person1.name, "George Orwell");
  person1.citNo = 1984;
  person1. salary = 2500;
  printf("Name: %s\n", person1.name);
  printf("Citizenship No.: %d\n", person1.citNo);
```

```
  printf("Salary: %.2f", person1.salary);
}
```

**Union:**

A union is a special data type available in C that allows to store different data types in the same memory location. You can define a union with many members, but only one member can contain a value at any given time. Unions provide an efficient way of using the same memory location for multiple-purpose.

***Syntax:***

```
union [union tag] {
   member definition;
   member definition;
   ...
   member definition;
} [one or more union variables];
```

***Example:***

```
union Data {
   int i;
   float f;
   char str[20];
} data;
```

***Program:***

```
union Data {
   int i;
   float f;
   char str[20];
};

void main( ) {
   union Data data;
   data.i = 10;
   data.f = 220.5;
   strcpy( data.str, "Welcome");
   printf( "data.i : %d\n", data.i);
   printf( "data.f : %f\n", data.f);
   printf( "data.str : %s\n", data.str);
}
```

**enum:**
Enumeration (or enum) is a user defined data type in C. It is mainly used to assign names to integral constants, the names make a program easy to read and maintain.

```
#include<stdio.h>

enum week{Mon, Tue, Wed, Thur, Fri, Sat, Sun};

void main()
{       enum week day;
        day = Wed;
        printf("%d",day);
 }
```

**Nested Structure:**
A nested structure in C is a structure within structure. One structure can be declared inside another structure in the same way structure members are declared inside a structure.

***Syntax:***

```
struct name_1
{
   member1;
  member2;
  .
  .
  membern;


  struct name_2
  {
     member_1;
     member_2;
     .
     .
    member_n;
  }, var1
```

} var2;

***Program:***

```
#include <stdio.h>
#include <string.h>

struct Employee
{
int employee_id;
char name[20];
int salary;
};

struct Organisation
{
char organisation_name[20];
char org_number[20];

struct Employee emp;
};

int main()
{
struct Organisation org;

printf("The size of structure organisation : %ld\n",sizeof(org));

org.emp.employee_id = 101;
strcpy(org.emp.name, "Mark");
org.emp.salary = 40000;
strcpy(org.organisation_name,"ABC");
strcpy(org.org_number, "12345678");


printf("Organisation Name : %s\n",org.organisation_name);
printf("Organisation Number : %s\n",org.org_number);
printf("Employee id : %d\n",org.emp.employee_id);
printf("Employee name : %s\n",org.emp.name);
printf("Employee Salary : %d\n",org.emp.salary);
}
```

**Array of Structures:**

An array of structure in C programming is a collection of different datatype variables, grouped together under a single name.

***The structural declaration is as follows −***

```
struct tagname{
  datatype member1;
  datatype member2;
  datatype member n;
};
```

***Example***

```
struct book{
  int pages;
  char author [30];
  float price;
};
```

An array of structres in C can be defined as the collection of multiple structures variables where each variable contains information about different entities. The array of structures in C are used to store information about multiple entities of different data types. The array of structures is also known as the collection of structures.

```
#include<stdio.h>
#include <string.h>

struct student
{
int rollno;
char name[10];
};

void  main()
{
int i;
struct student st[5];

printf("Enter Records of 5 students");
```

```
for(i=0;i<5;i++)
{
printf("\nEnter Rollno:");
scanf("%d",&st[i].rollno);
printf("\nEnter Name:");
scanf("%s",&st[i].name);
}

printf("\nStudent Information List:");
for(i=0;i<5;i++){
printf("\nRollno:%d, Name:%s",st[i].rollno,st[i].name);
}
}
```

**Self-referential Structures:**

The self-referential structure is a structure that points to the same type of structure. It contains one or more pointers that ultimately point to the same structure.

1. Structures are a user-defined data structure type in C and C++.
2. The main benefit of creating structure is that it can hold the different predefined data types.
3. It is initialized with the struct keyword and the structure's name followed by this struct keyword.
4. We can easily take the different data types like integer value, float, char, and others within the same structure.
5. It reduces the complexity of the program.
6. Using a single structure can hold the values and data set in a single place.
7. In the C++ programming language, placing struct keywords is optional or not mandatory, but it is mandatory in the C programming language.
8. Self-referential structure plays a very important role in creating other data structures like Linked list.
9. In this type of structure, the object of the same structure points to the same data structure and refers to the data types of the same structure.
10. It can have one or more pointers pointing to the same type of structure as their member.
11. The self-referential structure is widely used in dynamic data structures such as trees, linked lists, etc.

**Introduction to File:**

- A collection of data which is stored on a secondary device like a hard disk is known as a file.
- A file is generally used as real-life applications that contain a large amount of data.
- When a program is terminated, the entire data is lost. Storing in a file will preserve your data even if the program terminates.
- If you have to enter a large number of data, it will take a lot of time to enter them all.
- However, if you have a file containing all the data, you can easily access the contents of the file using a few commands in C.
- You can easily move your data from one computer to another without any changes.

**Types of Files:**

When dealing with files, there are two types of files you should know about:

- Text files
- Binary files

**1. Text files**
Text files are the normal .txt files. You can easily create text files using any simple text editors such as Notepad.

When you open those files, you'll see all the contents within the file as plain text. You can easily edit or delete the contents.

They take minimum effort to maintain, are easily readable, and provide the least security and takes bigger storage space.

**2. Binary files**
Binary files are mostly the .bin files in your computer.

Instead of storing data in plain text, they store it in the binary form (0's and 1's).

They can hold a higher amount of data, are not readable easily, and provides better security than text files.

**File Operations:**
In C, you can perform four major operations on files, either text or binary:

- Creating a new file
- Opening an existing file
- Closing a file
- Reading from and writing information to a file

**Declaration of file pointer:**
A pointer variable is used to points a structure FILE. The members of the FILE structure are used by the program in various file access operation, but programmers do not need to concerned about them.
FILE *file_pointer_name;
Eg : FILE *fp;

**Opening a file:**
To open a file using the fopen() function. Its syntax is,

FILE *fopen(const char *fname,const char* mode);
const char *fname represents the file name. The file name is like “example.txt”. const char *mode represents the mode of the file. It has the following values.

| **Mode** | **Description** |
|---|---|
| r | Opens a text file in reading mode |
| w | Opens or create a text file in writing mode |
| a | Opens a text file in append mode |
| r+ | Opens a text file in both reading and writing mode |
| w+ | Opens a text file in both reading and writing mode |
| a+ | Opens a binary file in reading mode |

Difference between write mode and append mode is, Write (w) mode and Append (a) mode, while opening a file are almost the same. Both are used to write in a file. In both the modes, new file is created if it doesn't exists already. The only difference they have is, when you open a file in the write mode, the file is reset, resulting in deletion of any data already present in the file. While in append mode this will not happen. Append mode is used to append or add data to the existing data of file(if any). Hence, when you open a file in Append(a) mode, the cursor is positioned at the end of the present data in the file.

**Closing and Flushing Files:**

The file must be closed using the fclose() function. Its prototype is , int fclose(FILE *fp);
The argument fp is the FILE pinter associated with the stream. Fclose() returns 0 on success and -1 on error.

**Detecting the end of a file:**
When reading from a text-mode file character by character, one can look for the end-of-file character. The symbolic constant EOF is defined in stdio.h as -1, a value never used by a real character.
Eg: while((c=getc(fp)!=EOF). This is the general representation of the End Of the File.

**Working with Text files:**
C provides various functions to working with text files. Four functions can be used to read text files and four functions that can be used to write text files into a disk. These are,
***fscanf() & fprintf()***
***fgets() & fputs()***
***fgetc() & fputc()***
***fread() & fwrite()***

The prototypes of the above functions are,
1. int fscanf(FILE *stream, const char *format,list);
2. int getc(FILE *stream);
3. char *fgets(char *str,int n,FILE *fp);
4. int fread(void *str,size_t size,size_t num, FILE *stream);
5. int fprintf(FILE *stream, const char *format,list);
6. int fputc(int c, FILE *stream);
7. int fputs(const char *str,FILE *stream);
8. int fwrite(const void *str,size_t size, size_t count,FILE *stream);

**Graphics functions in C.**

Graphics programming in C used to drawing various geometrical shapes(rectangle, circle eclipse etc), use of mathematical function in drawing curves, coloring an object with different colors and patterns and simple animation programs like jumping ball and moving cars. We can use graphics programming for developing your games, in making projects, for animation etc. It's not like traditional C programming in which you have to apply complex logic in your program.

The first step in any graphics program is to initialize the graphics drivers on the computer using initgraph method of graphics.h library.

***void initgraph(int *graphicsDriver, int *graphicsMode, char *driverDirectoryPath);***

It initializes the graphics system by loading the passed graphics driver then changing the system into graphics mode. It also resets or initializes all graphics settings like color, palette, current position etc, to their default values. Below is the description of input parameters of initgraph function.

**graphicsDriver :** It is a pointer to an integer specifying the graphics driver to be used. It tells the compiler that what graphics driver to use or to automatically detect the drive. In all our programs we will use DETECT macro of graphics.h library that instruct compiler for auto detection of graphics driver.

**graphicsMode :** It is a pointer to an integer that specifies the graphics mode to be used. If *graphdriver is set to DETECT, then initgraph sets *graphmode to the highest resolution available for the detected driver.

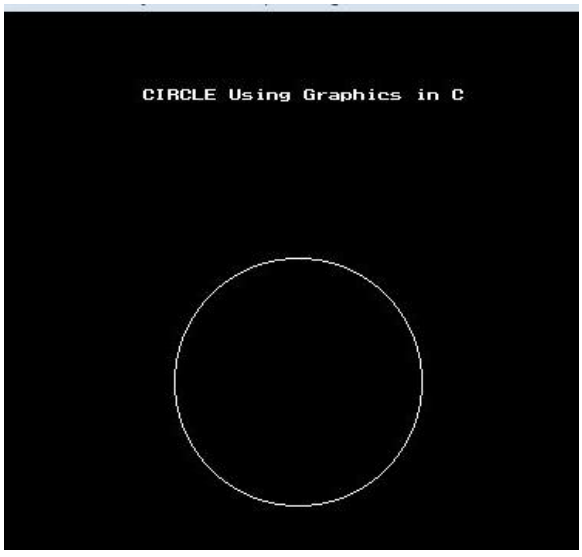
**driverDirectoryPath :** It specifies the directory path where graphics driver files (BGI files) are located. If directory path is not provided, then it will seach for driver files in current working directory directory. In all our sample graphics programs, you have to change path of BGI directory accordingly where you turbo C compiler is installed.

***Here is C Graphics program to draw a straight line on screen.***

```
/* C graphics program to draw a line */
#include<graphics.h>
#include<conio.h>
void main()
{
   int gd = DETECT, gm;
   /* initialization of graphic mode */
   initgraph(&gd, &gm, "C:\\TC\\BGI");
   line(100,100,200, 200);
   getch();
   closegraph();
}
```

In this program initgraph function auto detects an appropriate graphics driver and sets graphics mode maximum possible screen resolution. Then line function draws a straight line from coordinate (100, 100) to (200, 200). Then we added a call to getch function to avoid instant termination of program as it waits for user to press any key. At last, we unloads the graphics drivers and sets the screen back to text mode by calling closegraph function.

**C program to draw circle using c graphics**

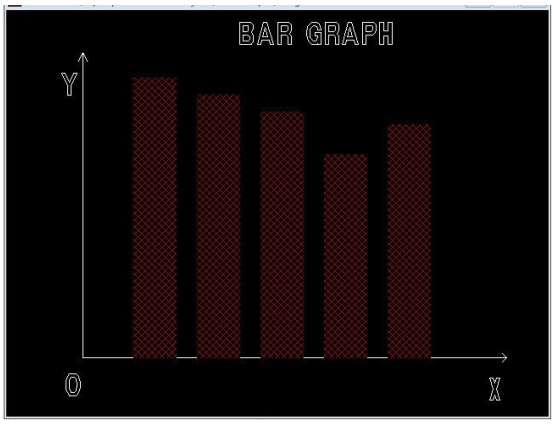



```
#include<stdio.h>
#include<graphics.h>

void main()
{
  int gd = DETECT,gm;
  int x ,y ,radius=80;
  initgraph(&gd, &gm, "C:\\TC\\BGI");
  x = getmaxx()/2;
  y = getmaxy()/2;

  outtextxy(x-100, 50, "CIRCLE Using Graphics in C");
  circle(x, y, radius);
  closegraph();
}
```

**C Program to Draw Bar Graph Using C Graphics**




```
#include <graphics.h>

void main() {
  int gd = DETECT, gm;
  initgraph(&gd, &gm, "X:\\TC\\BGI");

  settextstyle(BOLD_FONT,HORIZ_DIR,2);
  outtextxy(275,0,"BAR GRAPH");

  setlinestyle(SOLID_LINE,0,2);
  /* Draw X and Y Axis */
  line(90,410,90,50);
  line(90,410,590,410);
  line(85,60,90,50);
  line(95,60,90,50);
  line(585,405,590,410);
  line(585,415,590,410);

  outtextxy(65,60,"Y");
  outtextxy(570,420,"X");
  outtextxy(70,415,"O");
  /* Draw bars on screen */
  setfillstyle(XHATCH_FILL, RED);
  bar(150,80,200,410);
  bar(225,100,275,410);
  bar(300,120,350,410);
  bar(375,170,425,410);
  bar(450,135,500,410);

  closegraph();
}
```